

THERS

MANUAL OPERATIVO

De proyecto universitario a empresa tecnológica

Versión 1.0

Julio 2026

El Salvador

Documento interno — confidencial

Preparado como marco operativo, técnico y comercial del equipo THERS

Índice

Índice	2
Nota sobre el alcance y la vigencia de este documento	5
1. Introducción	6
1.1 Quiénes somos	6
1.2 Qué queremos lograr	6
1.3 Visión	6
1.4 Misión	6
1.5 Valores	6
1.6 Cultura tecnológica	6
2. Organización del equipo	8
2.1 Organigrama	8
2.2 Roles y responsabilidades	8
2.3 Cómo trabajaremos	8
2.4 Cómo tomaremos decisiones	8
2.5 Cómo documentaremos	8
2.6 Cómo haremos reuniones	9
3. Roadmap de 12 meses	10
3.1 Estructura semanal fija	10
3.2 Calendario mensual (temas y semanas)	10
3.3 Investigación semanal rotativa	10
4. Arquitectura de THERS	12
4.1 Visión general	12
4.2 Diagrama de flujo (texto)	12
4.3 Frontend	12
4.4 Backend	12
4.5 Base de datos	12
4.6 Docker y despliegue	13
4.7 Seguridad (resumen — detalle en Capítulo 10)	13
4.8 Escalabilidad	13
5. Herramientas	14
5.1 Tabla de referencia rápida	14
5.2 Herramientas de IA para desarrollo	14
5.3 Herramientas de colaboración y diseño	16
5.4 Infraestructura y despliegue	16
6. Presupuesto	17
6.1 Escenario estudiante (sin ingresos)	17
6.2 Escenario startup (primeros clientes)	17
6.3 Escenario empresa (equipo de 6–10, operación estable)	17
6.4 Escenario agencia (múltiples clientes simultáneos)	17
6.5 Escenario de escalamiento (crecimiento acelerado)	18
6.6 Qué comprar primero y qué puede esperar	18

7. Inteligencia artificial dentro de THERS	19
7.1 Mapa de funcionalidades de IA	19
7.2 RAG (Retrieval-Augmented Generation)	19
7.3 Agentes IA para el propio equipo	19
7.4 Qué LLM usar para cada tarea	19
8. Diseño de la empresa	21
8.1 Servicios	21
8.2 Nichos sugeridos	21
8.3 Portafolio y marketing	21
8.4 Ventas y consultoría	21
8.5 Forma legal de la empresa	21
9. Plan de SEO	22
9.1 Fundamentos técnicos (checklist)	22
9.2 Contenido y palabras clave	22
9.3 Cadencia recomendada	22
10. Seguridad	23
10.1 OWASP Top 10 aplicado a THERS	23
10.2 Autenticación	23
10.3 Protecciones específicas	23
10.4 Checklist de auditoría periódica	23
11. Dockerización de THERS paso a paso	25
11.1 Servicios a contenerizar	25
11.2 Pasos	25
11.3 Buenas prácticas	25
12. DevOps	26
12.1 Integración y despliegue continuo (CI/CD)	26
12.2 Estrategia de pruebas	26
12.3 Entornos y despliegue	26
12.4 Backups	26
12.5 Logs y monitoreo	26
13. Portafolio	27
13.1 Estructura de cada caso de estudio	27
13.2 Dónde vive el portafolio	27
14. Clientes	28
14.1 Cómo conseguir clientes	28
14.2 Cómo hacer propuestas	28
14.3 Cómo cotizar	28
14.4 Cómo vender	28
14.5 Cómo cerrar contratos y cobrar	28
15. Plan de crecimiento	29
16. Recomendaciones finales	30
16.1 Correcciones sobre la información recopilada	30
16.2 Riesgos a vigilar	30

16.3 Oportunidades..... 30

16.4 Próximos pasos sugeridos 30

16.5 Conclusión 31

Nota sobre el alcance y la vigencia de este documento

Este manual es la Versión 1.0 del sistema operativo de THERS como empresa. Sigue la recomendación de construir el manual "como un proyecto vivo, por versiones": esta primera versión cubre organización, roadmap, arquitectura, herramientas, presupuesto, IA, empresa, SEO, seguridad, Docker, DevOps, portafolio, clientes, plan de crecimiento y recomendaciones. Las versiones 2.0 y 3.0 profundizarán en implementación técnica detallada (código, configuraciones completas) y en estrategia comercial avanzada.

Sobre precios y planes:

Los precios de herramientas de IA y SaaS (ChatGPT, Claude, Cursor, GitHub Copilot, Figma, Cloudflare, Vercel, Railway, Render, AWS, Azure, etc.) cambian con frecuencia — a veces varias veces al año. Las cifras de este manual fueron verificadas en julio de 2026 contra fuentes públicas y sirven como referencia de orden de magnitud, no como cotización final. Antes de comprometer presupuesto, confirmen el precio vigente en el sitio oficial de cada herramienta.

Sobre el roadmap diario:

El documento original solicitaba un calendario de 12 meses desglosado día por día. En un manual de referencia, 365 líneas de tareas diarias no son mantenibles ni consultables: quedarían desactualizadas en la primera semana y no aportan más valor que un desglose semanal claro. Por eso el Capítulo 3 organiza el año en meses y semanas temáticas, con temas de estudio diario sugeridos (Lunes a Domingo) que se repiten como estructura fija. La asignación de tareas día a día debe vivir en una herramienta operativa (GitHub Projects o Notion), no en un documento estático — así el equipo la actualiza sin reeditar el manual.

1. Introducción

1.1 Quiénes somos

THERS nace como el proyecto de un equipo de estudiantes universitarios que decidió no tratarlo como una tarea de curso, sino como el primer producto de una empresa de desarrollo de software e inteligencia artificial. THERS es una red social construida con React, Vite, TailwindCSS, Node.js, Express, JWT y PostgreSQL, y es también el terreno de práctica donde el equipo aprende a trabajar con los procesos, la disciplina y la calidad de una empresa tecnológica real.

1.2 Qué queremos lograr

- Terminar THERS como un producto funcional, seguro, medible y presentable — no solo "que corra", sino que se pueda mostrar a un evaluador, un reclutador o un cliente.
- Instalar, desde el día uno, procesos de trabajo en equipo (Git, revisiones de código, documentación, reuniones) que se puedan reutilizar en cualquier proyecto futuro.
- Usar THERS como plataforma de aprendizaje guiado de las tecnologías que sostienen productos reales: contenedores, nube, CI/CD, seguridad, SEO e IA aplicada.
- Convertir el conocimiento acumulado en una empresa de servicios: desarrollo web, sistemas empresariales, IA, automatización y consultoría tecnológica.
- Construir un portafolio que hable por sí solo, mostrando no solo capturas de pantalla, sino el razonamiento detrás de cada decisión técnica.

1.3 Visión

Ser reconocidos, en un plazo de cinco años, como un equipo salvadoreño capaz de diseñar, construir y operar productos digitales y soluciones de inteligencia artificial con estándares profesionales, tanto para negocios locales como para clientes internacionales.

1.4 Misión

Diseñar y entregar software confiable, seguro y bien documentado, combinando desarrollo de alta calidad con inteligencia artificial aplicada, mientras formamos como profesionales capaces de sostener una empresa tecnológica propia.

1.5 Valores

Valor	Qué significa en la práctica
Excelencia técnica	Preferimos hacerlo bien una vez a corregirlo tres veces. Code review y estándares son obligatorios, no opcionales.
Aprendizaje continuo	Cada semana alguien del equipo investiga y expone un tema nuevo. Nadie se estanca.
Transparencia	Decisiones, errores y pendientes se documentan donde todo el equipo los vea (GitHub Issues, Notion), no en conversaciones privadas.
Orientación al cliente	Incluso en THERS, cada funcionalidad se evalúa por el problema real que resuelve, no solo por lo interesante que sea de programar.
Trabajo en equipo	El conocimiento no depende de una sola persona. Se documenta y se comparte.
Ética profesional	Seguridad, privacidad de datos y honestidad en compromisos con clientes van antes que la velocidad de entrega.

1.6 Cultura tecnológica

La cultura de THERS se apoya en cuatro hábitos no negociables:

1. Documentar mientras se construye, no después: cada módulo nuevo lleva su README o entrada en Notion antes de darse por terminado.
2. Revisar antes de fusionar: ningún cambio llega a la rama principal sin al menos una revisión de otro integrante.
3. Medir antes de optimizar: no se "mejora el rendimiento" sin datos (Lighthouse, consultas lentas, métricas reales) que lo respalden.
4. Actuar como proveedor, no como estudiante: cada entrega interna se trata como si un cliente externo la fuera a revisar.

2. Organización del equipo

2.1 Organigrama

Con un equipo pequeño (4–6 personas), un organigrama rígido no es realista ni útil. Se recomienda una estructura plana con responsabilidades claras y roles rotativos parciales, donde una persona puede cubrir más de un rol al inicio:

CEO / Product Manager → visión de producto, prioriza el backlog, habla con clientes

CTO / Software Architect → decisiones técnicas, arquitectura, estándares de código

└─ **Frontend Developer** → React, UI/UX, accesibilidad

└─ **Backend Developer** → API, autenticación, lógica de negocio

└─ **Database / DevOps Engineer** → PostgreSQL, Docker, CI/CD, despliegue

└─ **QA / Seguridad** → pruebas, OWASP, revisión de vulnerabilidades

UI/UX Designer → Figma, sistema de diseño, prototipos

Marketing / SEO → posicionamiento, contenido, analítica

2.2 Roles y responsabilidades

Rol	Responsabilidad principal	Entregable típico
CEO / PM	Visión, prioridades, relación con clientes	Backlog priorizado, roadmap
CTO / Arquitecto	Decisiones técnicas, calidad de código	ADRs, estándares, revisiones
Frontend	Interfaz, experiencia de usuario, consumo de API	Componentes React documentados
Backend	API REST, autenticación, reglas de negocio	Endpoints documentados (OpenAPI)
DB / DevOps	Modelo de datos, contenedores, CI/CD, despliegue	Esquema SQL, pipelines, entornos
QA / Seguridad	Pruebas, checklist OWASP, control de calidad	Reportes de prueba, checklist de seguridad
UI/UX	Wireframes, prototipos, sistema de diseño	Archivo Figma, guía de estilo
Marketing / SEO	Visibilidad, contenido, métricas	Reporte mensual de SEO/analítica

2.3 Cómo trabajaremos

Metodología ágil híbrida (Scrum + Kanban), adaptada a la disponibilidad real de un equipo de estudiantes:

- Sprints de 2 semanas, con objetivo claro por sprint.
- Tablero Kanban en GitHub Projects: Backlog → En progreso → En revisión → Hecho.
- Cada tarea es un Issue de GitHub, vinculado a una rama y a un Pull Request.
- Nadie trabaja directamente sobre main; todo pasa por rama de funcionalidad y revisión.

2.4 Cómo tomaremos decisiones

Las decisiones técnicas relevantes (elegir una librería, cambiar de arquitectura, adoptar una herramienta paga) se documentan como ADR (Architecture Decision Record): un archivo corto en el repositorio con el contexto, las opciones consideradas, la decisión tomada y las consecuencias. Esto evita repetir discusiones y deja rastro para quien se una al equipo después. Las decisiones de negocio (precios, clientes, alianzas) se registran en una bitácora de decisiones en Notion, con fecha y responsable.

2.5 Cómo documentaremos

Tipo de contenido	Dónde vive
Código, README, ADRs, Issues, Pull Requests	GitHub
Requisitos, casos de uso, base de conocimiento, actas de reunión	Notion
Especificación de la API	Swagger / OpenAPI
Diseño, prototipos, sistema de componentes	Figma
Propuestas, cotizaciones, contratos con clientes	Notion + carpeta compartida

2.6 Cómo haremos reuniones

Reunión	Frecuencia	Duración	Objetivo
Daily async (texto en Discord)	Diaria	5 min	Qué hice, qué haré, qué me bloquea
Sync semanal	Domingo	45–60 min	Revisar avance del sprint, resolver bloqueos
Exposición técnica	Semanal (rotativa)	20–30 min	Un integrante enseña al resto un tema nuevo
Retro de sprint	Cada 2 semanas	30 min	Qué funcionó, qué no, qué cambiar
Revisión de arquitectura	Mensual	45 min	Deuda técnica, rendimiento, seguridad

3. Roadmap de 12 meses

El roadmap se organiza en 12 meses con un tema central por mes y cuatro semanas temáticas dentro de cada mes. Cada semana tiene, además, una estructura diaria fija que se repite (ver 3.2) para que el aprendizaje y el desarrollo de THERS avancen en paralelo. La asignación fina de tareas por día vive en GitHub Projects / Notion, no en este documento.

3.1 Estructura semanal fija

Día	Enfoque
Lunes	Frontend (React, Tailwind, componentes)
Martes	Backend (Node, Express, API)
Miércoles	Base de datos / infraestructura (PostgreSQL, Redis, Docker)
Jueves	Testing y control de calidad
Viernes	Documentación y revisión de código
Sábado	Investigación individual (tema asignado de la semana)
Domingo	Reunión general: sync, retro y planeación de la siguiente semana

3.2 Calendario mensual (temas y semanas)

Mes	Tema central	Semanas
1	Fundamentos de desarrollo	S1 Git y GitHub · S2 React + Vite · S3 TailwindCSS · S4 Node.js + Express básico
2	Persistencia y contenedores	S1 PostgreSQL (modelado, SQL) · S2 JWT y autenticación · S3 Docker básico · S4 Docker Compose
3	Calidad y entorno profesional	S1 Testing (unitario) · S2 Linux esencial · S3 Nginx · S4 Cloudflare (DNS, CDN, WAF)
4	Automatización y arquitectura	S1 Git avanzado (rebase, flujos) · S2 GitHub Actions / CI básico · S3 Patrones de diseño · S4 SOLID y Clean Code
5	Arquitectura de software	S1 DDD (conceptos) · S2 Monolito vs. microservicios · S3 Redis (cache, sesiones) · S4 Integración: refactor de THERS con lo aprendido
6	Seguridad	S1 OWASP Top 10 · S2 XSS/CSRF/SQL Injection en THERS · S3 Rate limiting y Helmet · S4 Auditoría de seguridad de THERS
7	SEO y rendimiento	S1 Core Web Vitals y Lighthouse · S2 Search Console y sitemap · S3 Analytics 4 · S4 Optimización real de THERS
8	Cloud	S1 Fundamentos AWS · S2 Fundamentos Azure · S3 Despliegue en Railway/Render · S4 Comparativa y elección de proveedor
9	Inteligencia artificial I	S1 Fundamentos de LLMs y prompting · S2 RAG (conceptos y pgvector) · S3 Chatbot de soporte para THERS · S4 Moderación de contenido con IA
10	Inteligencia artificial II	S1 Recomendaciones y búsqueda semántica · S2 Agentes IA (Claude Code / Agent SDK) · S3 Automatización interna con IA · S4 Integración final en THERS
11	Escalabilidad y portafolio	S1 Introducción a Kubernetes (conceptual) · S2 Monitoreo y logs · S3 Documentar THERS como caso de portafolio · S4 Preparar demo y pitch
12	Empresa y clientes	S1 Definir servicios y precios · S2 Preparar propuestas y cotizaciones · S3 Primer contacto con cliente piloto · S4 Cierre de versión 1.0 del producto y del manual

3.3 Investigación semanal rotativa

Cada semana, un integrante distinto expone al equipo (20–30 min, sábado o domingo) un tema técnico o de negocio. Esto evita que el conocimiento dependa de una sola persona y refuerza lo cubierto en el calendario mensual. Ejemplos de temas para las primeras ocho semanas: Docker, JWT y seguridad de sesiones, Cloudflare, SEO técnico, fundamentos de IA aplicada, Linux esencial, fundamentos de AWS, Docker Compose y orquestación local.

Nota:

Este roadmap asume dedicación parcial (estudiantes con clases). Si el ritmo real del equipo es más lento, es preferible extender el calendario a 15–18 meses antes que saltarse los meses 6 (seguridad) y 9–10 (IA), que son los que más valor de portafolio y de negocio aportan.

4. Arquitectura de THERS

4.1 Visión general

THERS sigue una arquitectura de tres capas clásica, pensada para poder desplegarse barata y rápida al inicio, y escalar por partes cuando existan usuarios o clientes reales. No se recomienda arrancar con microservicios: con un equipo de 4–6 personas, un monolito bien organizado por módulos (auth, perfiles, publicaciones, comentarios, chat) es más rápido de construir, más fácil de depurar y perfectamente escalable durante la primera etapa.

4.2 Diagrama de flujo (texto)

Cliente (navegador / app)

↓

Cloudflare → DNS, CDN, protección DDoS/WAF, certificado TLS

↓

Frontend (React + Vite + TailwindCSS) → servido como estático (Vercel o Nginx)

↓ (fetch a la API)

Nginx (reverse proxy) → enruta a la API, TLS interno, balanceo si hay más de una instancia

↓

Backend (Node.js + Express) → autenticación JWT, lógica de negocio, validación

↓

↓

PostgreSQL (datos persistentes) **Redis** (sesiones, cache, colas)

↓

Almacenamiento de objetos (imágenes de perfil / publicaciones) — ej. S3 compatible

4.3 Frontend

- React + Vite para desarrollo rápido con recarga instantánea.
- TailwindCSS para consistencia visual sin escribir CSS a mano en cada componente.
- Estructura por features (auth/, feed/, perfil/, chat/) en lugar de por tipo de archivo — facilita que cada desarrollador trabaje en su módulo sin pisarse.
- Variables de entorno separadas para desarrollo, staging y producción.

4.4 Backend

- Node.js + Express, organizado en capas: rutas → controladores → servicios → acceso a datos.
- Autenticación con JWT de corta duración (access token) + refresh token de larga duración almacenado de forma segura (ver Capítulo 10).
- Validación de entrada en cada endpoint (por ejemplo con Zod o Joi) antes de tocar la base de datos.
- Documentación de la API con OpenAPI/Swagger desde el primer endpoint, no al final.

4.5 Base de datos

- PostgreSQL como base de datos principal: usuarios, publicaciones, comentarios, likes, relaciones de seguimiento.
- Redis para tres usos concretos: cache de consultas frecuentes, almacenamiento de sesiones/refresh tokens, y colas ligeras para tareas asíncronas (por ejemplo, procesar una imagen subida).

- Migraciones versionadas desde el primer día (no cambiar el esquema a mano en producción).

4.6 Docker y despliegue

Todo el sistema corre en contenedores desde desarrollo local hasta producción, para eliminar el clásico "en mi máquina sí funciona". El detalle paso a paso está en el Capítulo 11.

Entorno	Dónde vive	Cuándo usarlo
Desarrollo	Docker Compose en la máquina de cada integrante	Siempre, para trabajar en igualdad de condiciones
Staging / MVP	Vercel (frontend) + Railway o Render (backend + PostgreSQL + Redis)	Mientras no hay ingresos ni tráfico serio
Producción con clientes	AWS o Azure (contenedores gestionados) detrás de Cloudflare	Cuando el tráfico, el SLA o un cliente lo exijan

4.7 Seguridad (resumen — detalle en Capítulo 10)

- HTTPS en todas las capas, terminado en Cloudflare y renovado internamente con Nginx.
- JWT de corta vida + refresh tokens rotativos, revocables desde Redis.
- Cabeceras de seguridad con Helmet, limitación de tasa (rate limiting) en endpoints sensibles (login, registro).
- Contraseñas con bcrypt, nunca en texto plano ni en logs.

4.8 Escalabilidad

- Frontend: se sirve desde CDN (Cloudflare/Vercel), escala automáticamente sin esfuerzo adicional.
- Backend: sin estado (stateless) para poder correr varias instancias detrás de Nginx cuando el tráfico crezca.
- Base de datos: primero optimizar índices y consultas; solo después considerar réplicas de lectura.
- Cache: Redis reduce la carga a PostgreSQL en los endpoints más consultados (feed, perfiles).

5. Herramientas

Precios verificados en julio de 2026:

Estas cifras cambian con frecuencia (a veces cada trimestre). Úsenlas para presupuestar en orden de magnitud y confirmen el valor exacto en el sitio oficial de cada herramienta antes de pagar.

5.1 Tabla de referencia rápida

Herramienta	Categoría	Precio individual de referencia (jul. 2026)	Prioridad para THERS
ChatGPT Plus	IA generalista	US\$20/mes	Alta
Claude (Pro)	IA de desarrollo profundo	US\$20/mes (US\$17/mes anual) — incluye Claude Code	Muy alta
Claude Code	Agente de desarrollo	Incluido en Pro/Max/Team Premium/Enterprise	Muy alta
Cursor Pro	IDE con IA	US\$20/mes	Alta
GitHub Copilot Pro	Autocompletado con IA	US\$10/mes	Media
Gemini (Google)	IA generalista / multimodal	Variable según plan (revisar Google One AI)	Media
Perplexity	Búsqueda con IA	Plan gratuito suele bastar	Baja
Notion	Documentación y gestión	Gratis (equipos pequeños) / Plus desde ~US\$10/miembro/mes	Alta
Docker	Contenedores	Gratis para equipos pequeños	Muy alta
GitHub	Control de versiones	Gratis (Free/Team con límites generosos)	Muy alta
Figma	Diseño UI/UX	Gratis (Starter) / Professional ~US\$12–16 por editor/mes	Alta
Cloudflare	CDN, DNS, seguridad	Plan gratuito robusto / Pro desde ~US\$20/mes	Alta
Vercel	Hosting frontend	Gratis (Hobby) / Pro desde ~US\$20/mes	Alta
Railway	Hosting backend/DB	Uso medido con crédito inicial / planes desde ~US\$5–20/mes	Alta
Render	Hosting backend/DB	Plan gratuito limitado / planes pagos desde ~US\$7/mes	Media
AWS	Nube (IaaS/PaaS)	Pago por uso; capa gratuita los primeros 12 meses	Media (fase posterior)
Azure	Nube (IaaS/PaaS)	Pago por uso; créditos para estudiantes disponibles	Media (fase posterior)
Neon	PostgreSQL serverless	Plan gratuito generoso / planes pagos desde ~US\$19/mes	Alta
Supabase	Backend-as-a-service (Postgres + auth)	Plan gratuito / Pro desde ~US\$25/mes	Media
Redis (Upstash/Redis Cloud)	Cache y sesiones	Plan gratuito para volúmenes bajos	Alta
Nginx	Servidor / proxy reverso	Open source, gratis	Alta

5.2 Herramientas de IA para desarrollo

Claude y Claude Code

Claude (web, escritorio, móvil) y Claude Code comparten la misma suscripción: con un plan Pro, Max, Team Premium o Enterprise se accede a ambos desde un solo pago, compartiendo la misma cuota de uso. Claude Code no es un chatbot: es un agente que trabaja directamente sobre el repositorio — puede leer el proyecto completo, modificar código en varios archivos, ejecutar comandos, generar pruebas y documentar cambios. Para un equipo que ya trabaja en un repositorio real como THERS, esto es distinto a copiar y pegar código de un chat: se le puede pedir "revisa todo el backend y documenta las funciones sin comentarios" y el agente recorre el repositorio.

	Detalle
Ventajas	Trabaja sobre el proyecto completo, no solo sobre un fragmento; fuerte en arquitectura, refactorización y backend; buena calidad de razonamiento técnico
Desventajas	Consume cuota de uso rápido en sesiones largas; el plan Team Standard NO incluye Claude Code (se necesita Team Premium)
Cuándo comprarlo	Desde que el equipo empieza a tocar código real de THERS de forma constante
Cuándo NO	Si el uso es solo ocasional para dudas puntuales — ahí el plan gratuito puede alcanzar al inicio
Alternativas	Cursor (editor completo con IA), GitHub Copilot (autocompletado), ChatGPT (para explicación y arquitectura)

ChatGPT

Buen complemento a Claude para tareas de otro tipo: pensar en voz alta sobre arquitectura, modelar tablas y relaciones, explicar algoritmos, redactar documentación y roadmaps, e investigar y comparar tecnologías. No reemplaza a un agente que trabaje directo sobre el repositorio, pero es rápido para iterar ideas.

	Detalle
Ventajas	Muy versátil para redacción, planeación y explicación; ecosistema amplio de plugins/integraciones
Desventajas	No opera directamente sobre el repositorio como Claude Code; requiere copiar/pegar contexto
Cuándo comprarlo	Desde el inicio, para documentación, planeación y arquitectura
Cuándo NO	Si el equipo ya cubre esas necesidades con Claude y prefiere concentrar el presupuesto
Alternativas	Claude (chat), Gemini, Perplexity para investigación con fuentes

Cursor y GitHub Copilot

Cursor es un editor de código (fork de VS Code) construido alrededor de IA: entiende el proyecto completo y permite ediciones multi-archivo. GitHub Copilot es una extensión de autocompletado que se integra en el editor que ya se use, con integración muy fuerte con GitHub (issues, PRs, Actions).

	Cursor Pro	GitHub Copilot Pro
Precio de referencia	US\$20/mes	US\$10/mes
Fortaleza	Ediciones multi-archivo, contexto de todo el proyecto, modo agente	Autocompletado maduro, integración nativa con GitHub
Cuándo elegirlo	Si el equipo quiere un IDE completo orientado a IA	Si prefieren mantener su editor actual y solo añadir autocompletado

Gemini y Perplexity

Gemini (Google) es útil por su integración con el ecosistema Google y sus capacidades multimodales; Perplexity destaca para investigación con fuentes citadas en tiempo real. Para THERS, ambos son complementos opcionales: no sustituyen a Claude/Claude Code para desarrollo ni a ChatGPT para planeación, pero son buenos para investigación de mercado, tendencias y validación cruzada de información.

5.3 Herramientas de colaboración y diseño

- **GitHub** — obligatorio. No solo para guardar código: Issues, Pull Requests, Projects y Actions son la columna vertebral de cómo el equipo trabaja y de cómo se demuestra profesionalismo ante evaluadores o clientes.
- **Notion** — documentación viva: requisitos, casos de uso, manual técnico, sprints, pendientes. Cuando alguien pregunte "¿cómo organizaron el proyecto?", la respuesta es abrir Notion.
- **Figma** — no Canva. Figma permite prototipos, componentes reutilizables, diseño responsivo y un sistema de color y tipografía consistente — como lo haría un equipo de producto real.
- **Discord** — llamadas, pantalla compartida y comunicación async fuera de las reuniones formales.

5.4 Infraestructura y despliegue

Recomendación por etapas: empezar en los niveles gratuitos o de entrada de Vercel (frontend), Railway o Render (backend + base de datos) y Neon o Supabase (PostgreSQL gestionado), y migrar a AWS o Azure solo cuando el tráfico, un contrato con SLA o requisitos de un cliente lo justifiquen. Empezar directamente en AWS/Azure sin necesidad añade complejidad operativa (redes, IAM, costos variables) que un equipo pequeño no necesita todavía.

Herramienta	Para qué sirve	Cuándo NO comprarla todavía
Cloudflare	DNS, CDN, protección contra DDoS, WAF, certificados TLS	El plan gratuito cubre casi todas las necesidades iniciales
Vercel	Hosting y despliegue continuo del frontend	Mientras el tráfico sea bajo, el plan gratuito alcanza
Railway / Render	Hosting de backend y base de datos con despliegue simple	No paguen el nivel más alto sin necesidad real de recursos
AWS / Azure	Nube completa (cómputo, almacenamiento, redes, IA gestionada)	No lo usen como primer despliegue: la curva de configuración no compensa para un MVP
Neon / Supabase	PostgreSQL gestionado (y auth/backend en el caso de Supabase)	Si ya administran su propio PostgreSQL en Railway/Render sin fricción
Redis (Upstash/Redis Cloud)	Cache, sesiones, colas ligeras	Si el tráfico aún es bajo, puede posponerse un mes o dos
Nginx	Proxy reverso, balanceo, terminación TLS interna	Prácticamente nunca — es gratis y es buena práctica desde el inicio

6. Presupuesto

Las siguientes tablas son estimaciones en dólares estadounidenses, con precios de referencia de julio de 2026 (ver disclaimer del Capítulo 5). Se asume un equipo base de 4 integrantes, ajustable según el escenario.

6.1 Escenario estudiante (sin ingresos)

Herramienta	Costo mensual	Costo por integrante (4)
Claude Pro (1 licencia compartida por el CTO/lead técnico)	US\$20	US\$5
ChatGPT Plus (1 licencia)	US\$20	US\$5
GitHub, Docker, Figma (Free), Cloudflare (Free), Vercel (Free)	US\$0	US\$0
Total	US\$40/mes	US\$10/mes

6.2 Escenario startup (primeros clientes)

Herramienta	Costo mensual
Claude Pro x2 (CTO + lead backend)	US\$40
ChatGPT Plus x1	US\$20
Cursor Pro x1 (opcional)	US\$20
Figma Professional (2 editores)	US\$24–32
Railway/Render (backend + DB en producción ligera)	US\$15–25
Dominio (prorrateado)	US\$1–2
Total aproximado	US\$120–140/mes

Costo anual aproximado: US\$1,440–1,680. Retorno esperado: con dos proyectos de cliente de US\$250–350/mes cada uno (o proyectos de alcance fijo equivalentes), la inversión en herramientas queda cubierta y el resto se reinvierte en crecimiento del equipo.

6.3 Escenario empresa (equipo de 6–10, operación estable)

Herramienta	Costo mensual
Claude Pro/Max según carga (2–3 licencias)	US\$60–150
ChatGPT Plus (2 licencias)	US\$40
Cursor Pro o Copilot Pro (todo el equipo dev)	US\$50–100
Figma Professional (equipo de diseño)	US\$50–80
Notion Plus	US\$60–100
Infraestructura (Vercel/Railway/Neon en niveles pagos)	US\$80–150
Cloudflare Pro	US\$20
Total aproximado	US\$360–640/mes

6.4 Escenario agencia (múltiples clientes simultáneos)

Rubro	Costo mensual estimado
Licencias de IA para todo el equipo (Claude + Cursor/Copilot)	US\$300–500

Rubro	Costo mensual estimado
Infraestructura por cliente (staging + producción)	US\$100–300 por cliente activo
Herramientas de gestión (Notion Business, Figma Organization)	US\$150–250
Monitoreo y seguridad (Sentry, escaneo de vulnerabilidades)	US\$50–150
Total aproximado (3 clientes activos)	US\$900–1,600/mes

6.5 Escenario de escalamiento (crecimiento acelerado)

Cuando THERS o los proyectos de clientes empiezan a tener tráfico serio, los costos migran de "licencias por persona" a "infraestructura por uso": cómputo en AWS/Azure, bases de datos con réplicas, CDN con mayor volumen, monitoreo avanzado y, eventualmente, un ingeniero dedicado a DevOps/Cloud. En esta etapa el presupuesto deja de ser fijo y pasa a evaluarse como porcentaje de ingresos (una referencia común en la industria es mantener la infraestructura entre 5% y 15% de los ingresos, ajustando según el margen del negocio).

6.6 Qué comprar primero y qué puede esperar

Comprar primero	Puede esperar
GitHub, Docker, Git (gratis, imprescindibles desde el día 1)	AWS/Azure (esperar hasta necesitar escalar de verdad)
Claude Pro para quien lidera el desarrollo	Cursor Pro para todo el equipo (empezar con 1 licencia y evaluar)
Cloudflare Free + Vercel Free + Railway/Render nivel de entrada	Figma Organization / Enterprise (el plan Professional alcanza mucho tiempo)
Un dominio propio para THERS	Herramientas de SEO avanzadas tipo Ahrefs (Search Console + Analytics, gratis, cubren el inicio)

7. Inteligencia artificial dentro de THERS

La IA en THERS debe diseñarse como funcionalidad de producto, no como un experimento aislado. Se recomienda introducirla por capas, en el orden en que aporta más valor con menos riesgo.

7.1 Mapa de funcionalidades de IA

Funcionalidad	Qué resuelve	Enfoque técnico sugerido
Moderación de contenido	Detectar contenido tóxico, spam o inapropiado antes de publicarse	Clasificador de texto (y de imagen si aplica) como paso previo a guardar la publicación
Buscador inteligente	Encontrar publicaciones o perfiles por significado, no solo por palabra exacta	Búsqueda semántica con embeddings + pgvector sobre PostgreSQL
Recomendaciones de feed	Mostrar contenido relevante a cada usuario	Filtrado híbrido: señales de interacción + similitud por embeddings
Chatbot de soporte	Responder dudas frecuentes de usuarios	RAG sobre la base de conocimiento/FAQ de THERS
Análisis de usuarios	Entender engagement y detectar abandono temprano	Panel de métricas + modelos simples de análisis de tendencia
Automatización interna	Acelerar el trabajo del propio equipo	Agentes IA para revisión de código, resúmenes de PRs, generación de pruebas

7.2 RAG (Retrieval-Augmented Generation)

Para el chatbot de soporte y el buscador inteligente, el patrón recomendado es RAG: en lugar de que el modelo "invente" respuestas, se recuperan primero los fragmentos de información relevantes (FAQ, publicaciones, perfiles) y se le entregan al modelo como contexto para responder. Flujo simplificado:

Pregunta del usuario → generar embedding → buscar los fragmentos más similares en pgvector → construir el prompt con esos fragmentos → LLM genera la respuesta final

7.3 Agentes IA para el propio equipo

Además de la IA de cara al usuario final, el equipo puede usar agentes (como Claude Code) para tareas internas de mantenimiento: detectar código duplicado, encontrar consultas SQL lentas, señalar funciones sin usar, revisar dependencias desactualizadas y verificar que el código respeta SOLID y Clean Code. Se recomienda una revisión semanal (por ejemplo cada viernes) donde el agente responde explícitamente:

- ¿Qué módulos tienen código duplicado?
- ¿Qué consultas SQL son lentas?
- ¿Qué archivos están demasiado grandes o hacen demasiadas cosas?
- ¿Qué vulnerabilidades o dependencias obsoletas existen?
- ¿Qué partes del código violan SOLID o buenas prácticas de Clean Code?

7.4 Qué LLM usar para cada tarea

Tarea	Herramienta recomendada	Por qué
Refactorización y trabajo directo sobre el repositorio	Claude Code	Opera sobre el proyecto completo, no solo un fragmento de texto
Arquitectura, modelado de datos, explicación de conceptos	ChatGPT o Claude (chat)	Buen razonamiento conversacional y de planeación

Tarea	Herramienta recomendada	Por qué
Autocompletado mientras se escribe código	GitHub Copilot o Cursor Tab	Integrado en el flujo de escritura, baja fricción
Moderación / clasificación en producción	Modelo especializado más pequeño vía API, no un chat manual	Costo y latencia predecibles para tráfico real
Búsqueda semántica y recomendaciones	Modelo de embeddings vía API + pgvector	Es un problema de similitud vectorial, no de conversación

Importante:

La moderación de contenido y las recomendaciones de producción no deben depender de copiar y pegar texto en un chat: se integran vía API, con límites de costo y tiempo de respuesta definidos desde el diseño.

8. Diseño de la empresa

THERS es el producto insignia; la empresa que lo respalda debe poder vivir de servicios a clientes reales desde el primer año. Este capítulo define esa capa comercial.

8.1 Servicios

- Desarrollo web (sitios institucionales, tiendas en línea, plataformas a medida).
- Desarrollo de sistemas empresariales (gestión interna, inventario, facturación).
- Inteligencia artificial aplicada (chatbots, automatización de procesos, análisis de datos).
- Automatización de tareas repetitivas (integraciones, flujos de trabajo).
- Consultoría tecnológica (arquitectura, elección de stack, auditoría técnica).
- SEO y optimización de rendimiento web.

8.2 Nichos sugeridos

Para un equipo que inicia en El Salvador/Centroamérica, conviene enfocar la búsqueda de clientes en nichos donde el ciclo de venta es corto y el dolor es concreto: pequeños y medianos comercios sin presencia digital seria, clínicas y consultorios que necesitan agendamiento y presencia en línea, restaurantes y negocios de comida con pedidos, y emprendimientos de moda/belleza que ya venden por redes sociales pero sin una plataforma propia. Este último nicho conecta directamente con el know-how del equipo en e-commerce (por ejemplo, proyectos de tienda en línea ya desarrollados por el equipo).

8.3 Portafolio y marketing

El portafolio (ver Capítulo 13) es la principal herramienta de marketing al inicio: sin presupuesto de pauta, el crecimiento viene de mostrar trabajo real bien documentado, presencia activa en redes profesionales (LinkedIn) y locales, y referidos de los primeros clientes satisfechos.

8.4 Ventas y consultoría

El proceso comercial recomendado tiene cuatro pasos: (1) diagnóstico gratuito breve del problema del cliente, (2) propuesta escrita con alcance y precio claros, (3) desarrollo con checkpoints visibles para el cliente, (4) entrega con documentación y una breve capacitación de uso. Este proceso, documentado una sola vez, se reutiliza con cada cliente nuevo (ver Capítulo 14).

8.5 Forma legal de la empresa

Nota legal:

La elección de forma societaria (empresa individual, sociedad de responsabilidad limitada u otra) y las obligaciones fiscales en El Salvador dependen de factores específicos del equipo y cambian con la normativa vigente. Este manual no sustituye asesoría legal o contable; se recomienda consultar con un contador público o abogado antes de facturar formalmente a clientes.

9. Plan de SEO

9.1 Fundamentos técnicos (checklist)

- Alta y verificación del sitio en Google Search Console (indexación, errores de rastreo, rendimiento en búsqueda).
- Google Analytics 4 configurado con eventos clave (registro, publicación, interacción).
- Auditorías periódicas con Lighthouse (rendimiento, accesibilidad, SEO, buenas prácticas).
- Métricas de Core Web Vitals (LCP, INP, CLS) monitoreadas, no solo medidas una vez.
- Sitemap.xml generado y enviado a Search Console.
- robots.txt configurado correctamente (permitir rastreo de páginas públicas, bloquear rutas privadas).
- Marcado Schema.org relevante (Organization, SocialMediaPosting/Person según la página).
- Etiquetas Open Graph y Twitter Card para que los enlaces compartidos se vean bien en redes.

9.2 Contenido y palabras clave

Para THERS como red social, el SEO no compete solo por "palabras clave de producto": las páginas públicas de perfil y contenido son las que más tráfico orgánico pueden generar si están bien estructuradas (URLs limpias, metadatos por página, tiempos de carga bajos). Para la empresa, el contenido debe orientarse a las búsquedas que hace un cliente potencial en el nicho elegido (por ejemplo, "desarrollo web para pymes El Salvador").

9.3 Cadencia recomendada

Actividad	Frecuencia
Revisión de Search Console (errores, indexación)	Semanal
Revisión de Analytics (usuarios, comportamiento)	Semanal
Auditoría Lighthouse completa	Mensual
Revisión y actualización de contenido/palabras clave	Mensual
Auditoría de Core Web Vitals en producción	Mensual

Sobre herramientas de pago como Ahrefs:

Google Search Console y Analytics 4 son gratuitos y cubren la mayor parte de las necesidades iniciales. Una herramienta paga de investigación de palabras clave y backlinks solo se justifica cuando el SEO se convierte en un servicio que se vende a clientes, no antes.

10. Seguridad

10.1 OWASP Top 10 aplicado a THERS

Riesgo OWASP	Cómo se manifiesta en THERS	Mitigación
Control de acceso roto	Un usuario edita o borra contenido de otro usuario	Verificar propiedad del recurso en cada endpoint, no solo autenticación
Fallas criptográficas	Contraseñas o tokens mal almacenados	bcrypt para contraseñas, HTTPS en todo el tráfico, secretos fuera del código
Inyección (SQL, etc.)	Consultas construidas con texto concatenado	Consultas parametrizadas / ORM, nunca interpolar entrada del usuario
Diseño inseguro	Falta de límites de negocio (ej. sin límite de publicaciones por minuto)	Definir reglas de negocio y validarlas en el backend, no solo en el frontend
Configuración incorrecta	Cabeceras de seguridad ausentes, CORS demasiado abierto	Helmet, configuración explícita de CORS, variables de entorno por ambiente
Componentes vulnerables	Dependencias de npm desactualizadas	Auditoría periódica (npm audit) y actualización controlada
Fallas de autenticación	Tokens de vida muy larga, sin rotación	JWT de corta duración + refresh tokens rotativos y revocables
Fallas de integridad de datos	Datos manipulados en tránsito o por dependencias no verificadas	TLS de extremo a extremo, verificación de paquetes
Registro y monitoreo insuficientes	No se detectan ataques ni errores a tiempo	Logs estructurados + monitoreo de errores (ver Capítulo 12)
Falsificación de solicitudes del lado del servidor (SSRF)	El backend hace peticiones a URLs controladas por el usuario	Lista blanca de dominios/IPs permitidos para peticiones salientes

10.2 Autenticación

- Access token JWT de corta duración (minutos), no de días.
- Refresh token de mayor duración, almacenado de forma segura (cookie httpOnly + Secure) y revocable desde Redis.
- Rotación de refresh tokens en cada uso, para detectar reutilización sospechosa.
- Contraseñas con bcrypt (costo adecuado, nunca MD5/SHA1 sin salting).

10.3 Protecciones específicas

Amenaza	Mitigación en THERS
XSS	Sanitizar entrada y salida de contenido generado por usuarios; Content Security Policy vía Helmet
CSRF	Tokens anti-CSRF en formularios sensibles, cookies SameSite
SQL Injection	Consultas parametrizadas / ORM, nunca concatenación de strings
Fuerza bruta en login	Rate limiting por IP y por usuario, bloqueo temporal tras varios intentos fallidos
Fuga de datos en logs	Nunca registrar contraseñas, tokens completos ni datos personales sensibles

10.4 Checklist de auditoría periódica

- ¿Todos los endpoints validan que el usuario autenticado tiene permiso sobre el recurso solicitado?
- ¿Existen dependencias con vulnerabilidades conocidas sin actualizar?
- ¿Las variables sensibles (claves, secretos) están fuera del control de versiones?

- ¿Los mensajes de error exponen información interna (rutas, stack traces) en producción?
- ¿Existen límites de tasa en endpoints críticos (login, registro, recuperación de contraseña)?

11. Dockerización de THERS paso a paso

Objetivo: que cualquier integrante del equipo levante todo el sistema (frontend, backend, base de datos, cache y proxy) con un solo comando, en igualdad de condiciones.

11.1 Servicios a contenerizar

Servicio	Imagen base sugerida	Rol
frontend	node (build) → nginx (servir estático)	Compila React/Vite y sirve los archivos estáticos
backend	node:LTS	API Express
db	postgres:LTS	Base de datos principal
cache	redis:LTS	Cache, sesiones, colas
proxy	nginx	Reverse proxy hacia frontend y backend

11.2 Pasos

1. Crear un Dockerfile en /backend con una etapa: instalar dependencias con npm ci, copiar el código y exponer el puerto de la API.
2. Crear un Dockerfile en /frontend con dos etapas: una etapa 'build' que corre npm run build con Vite, y una etapa final que copia los archivos generados a una imagen de Nginx liviana.
3. Definir variables de entorno por servicio (.env.development, .env.production) y nunca incluirlas en el repositorio — usar un .env.example como referencia.
4. Escribir docker-compose.yml en la raíz del proyecto, declarando los cinco servicios (frontend, backend, db, cache, proxy), sus puertos, variables de entorno y dependencias de arranque (depends_on).
5. Configurar un volumen persistente para PostgreSQL (para no perder datos al reiniciar contenedores) y, opcionalmente, uno para Redis.
6. Definir una red interna de Docker Compose para que los servicios se hablen por nombre (por ejemplo, el backend se conecta a 'db' y 'cache', no a 'localhost').
7. Configurar Nginx como proxy reverso: rutas /api hacia el backend, el resto hacia el frontend.
8. Probar localmente con 'docker compose up --build' y verificar que las cinco piezas levantan y se comunican correctamente.
9. Documentar en el README los comandos básicos (levantar, apagar, ver logs, entrar a un contenedor) para que cualquier integrante nuevo pueda arrancar el proyecto sin ayuda.
10. Conectar el mismo docker-compose (o una variante) al pipeline de CI/CD para pruebas automatizadas y, más adelante, al despliegue (ver Capítulo 12).

11.3 Buenas prácticas

- Usar imágenes '-alpine' o slim cuando sea posible, para builds más rápidos y livianos.
- No correr los contenedores como usuario root en producción.
- Separar el docker-compose de desarrollo (con hot-reload) del de producción (imágenes optimizadas, sin herramientas de desarrollo).
- Nunca hornear (hardcodear) secretos dentro de la imagen: siempre inyectarlos como variables de entorno en tiempo de ejecución.

12. DevOps

12.1 Integración y despliegue continuo (CI/CD)

Con GitHub Actions, cada Pull Request dispara automáticamente un flujo que evita que código roto o de baja calidad llegue a la rama principal:

Pull Request abierto

↓

Lint (estilo de código) → Pruebas unitarias → Build (frontend y backend)

↓

¿Todo pasó? Sí → habilita el merge | No → bloquea el merge y notifica

↓ (al hacer merge a main)

Despliegue automático a staging → verificación manual → despliegue a producción

12.2 Estrategia de pruebas

Tipo de prueba	Qué cubre	Cuándo corre
Unitarias	Funciones y lógica aisladas (validaciones, utilidades)	En cada commit / PR
Integración	Endpoints de la API contra una base de datos de prueba	En cada PR
End-to-end (E2E)	Flujos completos de usuario (registro, publicar, comentar)	Antes de cada despliegue a producción

12.3 Entornos y despliegue

- Ambiente de staging que refleja producción, usado para validar antes de liberar.
- Despliegue a producción solo desde la rama principal, nunca desde ramas de funcionalidad.
- Rollback documentado: capacidad de volver a la versión anterior si algo falla tras un despliegue.

12.4 Backups

- Respaldo automático diario de PostgreSQL (pg_dump programado o el respaldo gestionado del proveedor de hosting).
- Retención mínima de 7 días de respaldos, con al menos una copia fuera del proveedor principal.
- Prueba periódica de restauración: un respaldo que nunca se ha restaurado no es un respaldo confiable.

12.5 Logs y monitoreo

- Logs estructurados (formato JSON) en el backend, con nivel (info, warning, error) y sin datos sensibles.
- Monitoreo de errores en producción (por ejemplo Sentry) para enterarse de fallas antes que los usuarios las reporten.
- Monitoreo de disponibilidad (uptime) de frontend y API.
- Alertas automáticas (Discord/correo) cuando la tasa de errores supera un umbral definido.

13. Portafolio

Un portafolio de capturas de pantalla no demuestra criterio técnico. Cada proyecto (THERS incluido) debe documentarse como caso de estudio, siguiendo la misma estructura siempre, para que un evaluador o un cliente pueda comparar proyectos entre sí.

13.1 Estructura de cada caso de estudio

1. Problema — qué necesidad real motivó el proyecto.
2. Análisis — qué opciones se consideraron y por qué se descartaron algunas.
3. Arquitectura — diagrama y decisiones clave (con enlace a los ADR si existen).
4. Solución — qué se construyó, con foco en las decisiones no obvias.
5. Tecnologías — stack completo, justificado (no solo listado).
6. Resultados — métricas concretas cuando existan (rendimiento, tiempo de carga, usuarios).
7. Impacto — qué cambió para el usuario o el cliente gracias al proyecto.
8. Aprendizajes — qué se haría distinto la próxima vez.

13.2 Dónde vive el portafolio

Recomendado: una página propia de la empresa (además del perfil de GitHub), con una entrada por proyecto siguiendo la estructura anterior, y enlaces al repositorio y, si aplica, a una demo en vivo.

14. Clientes

14.1 Cómo conseguir clientes

- Red de contacto cercana primero: familia, conocidos con negocio, profesores con vínculos empresariales.
- Presencia activa en LinkedIn mostrando el proceso de construcción de THERS, no solo el resultado final.
- Participar en eventos y comunidades tecnológicas locales.
- Pedir referidos explícitamente a cada cliente satisfecho — es la fuente más barata y de mejor calidad.

14.2 Cómo hacer propuestas

Toda propuesta debe responder, en una página, cuatro preguntas: qué problema resuelve, qué se va a construir (alcance explícito, incluyendo qué NO incluye), cuánto tiempo tomará y cuánto cuesta. Los alcances ambiguos son la causa número uno de conflictos con clientes.

14.3 Cómo cotizar

Modelo	Cuándo usarlo
Precio fijo por alcance definido	Proyectos con requisitos claros (sitio institucional, landing, integración puntual)
Por hora/día	Trabajo de consultoría o soporte donde el alcance es variable
Retainer mensual	Mantenimiento continuo, soporte o mejora incremental de un producto ya en producción

14.4 Cómo vender

Estructura sugerida para la primera llamada con un cliente potencial: (1) entender el problema antes de hablar de tecnología, (2) reformular el problema con sus propias palabras para confirmar que se entendió, (3) explicar en términos simples cómo se resolvería, (4) comprometerse solo a enviar una propuesta por escrito, nunca a un precio improvisado en la llamada.

14.5 Cómo cerrar contratos y cobrar

- Contrato o acuerdo simple por escrito, incluso para proyectos pequeños: alcance, precio, forma de pago y fechas.
- Anticipo antes de iniciar (habitualmente 30–50%) para cubrir costos y confirmar compromiso real del cliente.
- Pagos ligados a hitos verificables (por ejemplo: anticipo, entrega de MVP, entrega final) en vez de un solo pago al final.
- Facturación clara y puntual; llevar un registro simple de cuentas por cobrar.

Nota legal:

Los contratos con clientes deben adaptarse a la legislación salvadoreña vigente. Este manual describe buenas prácticas generales, no asesoría legal.

15. Plan de crecimiento

Horizonte	Qué deben dominar	Qué debe existir en la empresa
3 meses	React, Node/Express, Git/GitHub, JWT básico, PostgreSQL básico	THERS con autenticación y funcionalidades core funcionando
6 meses	Docker/Docker Compose, testing, CI básico, SOLID/Clean Code	THERS dockerizado, con CI y primeras pruebas automatizadas
12 meses	Seguridad OWASP, SEO técnico, fundamentos de IA aplicada (RAG, agentes)	THERS en producción con IA integrada; primer cliente piloto
24 meses	Cloud (AWS/Azure), Kubernetes conceptual, arquitectura para escalar	2–3 clientes activos, procesos de venta y entrega repetibles
36 meses	Especialización de cada integrante (uno fuerte en cloud, otro en IA, etc.)	Empresa con ingresos estables, portafolio consolidado, posible primera contratación

16. Recomendaciones finales

16.1 Correcciones sobre la información recopilada

- **Claude Pro y Claude Code:** es correcto que comparten suscripción (Pro, Max, Team Premium o Enterprise incluyen ambos bajo una misma cuota). La precisión que faltaba: el plan Team Standard NO incluye Claude Code, solo Team Premium sí. Y, como con cualquier herramienta de IA, los planes y límites de uso cambian con cierta frecuencia — conviene reconfirmar en support.claude.com antes de presupuestar.
- **Calendario diario de 12 meses:** pedir un desglose día por día durante un año no es sostenible como documento de referencia (quedaría desactualizado casi de inmediato). Se sustituyó por un calendario mensual con semanas temáticas y una estructura diaria fija reutilizable (Capítulo 3), que es lo que realmente se puede mantener actualizado.
- **Sobreinversión temprana en herramientas:** varias de las propuestas originales (licencias completas de Cursor y ChatGPT para todo el equipo, Figma Organization, Ahrefs) tienen sentido cuando ya hay ingresos, no antes. Se reordenó la prioridad de compra en el Capítulo 6.6.
- **Arquitectura de microservicios prematura:** con 4–6 personas y un producto que aún no tiene usuarios reales, un monolito modular es más rápido de construir y depurar que microservicios. Migrar a microservicios es una decisión que se toma cuando un módulo específico realmente lo necesita, no por defecto.

16.2 Riesgos a vigilar

- Dependencia de una sola persona para conocimiento crítico — mitigado por la investigación semanal rotativa y la documentación obligatoria.
- Rotación típica de equipos universitarios (alguien se retira o pierde disponibilidad) — mitigado documentando desde el día uno, no cuando ya es urgente.
- Scope creep con los primeros clientes (aceptar cambios de alcance sin ajustar precio ni tiempo) — mitigado con propuestas por escrito y control de cambios.
- Deuda técnica de seguridad acumulada por priorizar features sobre OWASP — mitigado con la auditoría periódica del Capítulo 10.4.
- Agotamiento (burnout) por combinar estudios, THERS y clientes — vigilar la carga real del equipo en las retros de sprint.

16.3 Oportunidades

- Vacío de mercado en pymes locales sin presencia digital seria — nicho accesible y de ciclo de venta corto.
- Posicionarse desde el inicio como "equipo que integra IA de verdad" (no solo como palabra de moda) es un diferenciador real frente a agencias tradicionales.
- El propio proceso de construir THERS con procesos profesionales es, en sí mismo, material de marketing (mostrar el camino, no solo el resultado).

16.4 Próximos pasos sugeridos

1. Definir el motor de base de datos si aún no está decidido (PostgreSQL, según el stack ya listado) y dejarlo documentado como ADR.
2. Configurar GitHub Projects con el tablero Kanban descrito en el Capítulo 2.
3. Levantar el entorno con Docker Compose siguiendo el Capítulo 11 antes de seguir agregando funcionalidades nuevas.
4. Programar la primera auditoría de seguridad básica (Capítulo 10.4) antes de exponer THERS públicamente.
5. Planear la versión 2.0 de este manual: desarrollo técnico detallado (código, configuraciones completas) y estrategia comercial avanzada.

16.5 Conclusión

La diferencia entre un proyecto escolar y el primer producto de una empresa no está en la tecnología usada, sino en los procesos: documentar, revisar, medir y tratar cada entrega como si un cliente real la fuera a evaluar. Este manual es el punto de partida de esos procesos para THERS. Su valor no está en quedar terminado, sino en usarse cada semana y actualizarse a medida que el equipo aprende — tal como se planteó desde el inicio: organización, herramientas, costos, roadmap y arquitectura primero (v1.0); desarrollo, Docker, CI/CD, seguridad y SEO después (v2.0); IA, automatización y estrategia comercial al final (v3.0).